Localization and Mapping with MacAEV

ELECENG 3EY4: Electrical Systems Integration Project

Instructor: Shahin Sirouspour

Objective: The project activities in the next three weeks will focus on localizing the vehicle and mapping its surroundings. We'll use a combination of on-board LiDAR scan data, vehicle velocity, and steering angle, as well as IMU measurements, for this task. First, we'll integrate the vehicle's kinematic motion equations based on the VESC-reported motor speed and steering angle to determine the vehicle's pose relative to a fixed odometry frame; this is known as dead reckoning using wheel odometry. We'll modify the standard wheel odometry to utilize the IMU yaw measurement instead of integration for calculating the vehicle's angular position. Subsequently, we'll implement an *Occupancy Grid Mapping* algorithm that employs the vehicle's pose from the wheel odometry and LiDAR scan data to construct a map of the vehicle's surrounding environment.

Safety Instructions:

- Refer to the safety instructions on the course Avenue to Learn website and the course outline before attempting to carry out the activities set out in this document.
- The AEV must never by powered the LiPo battery except in the lab and under the supervision of a course TA.
- You must never remove the LiPo batteries from the lab rooms.
- You must use the wall-mounted switching power supply to power the AEV for activities that can be performed outside the regular laboratory hours at home.
- When driving the AEV, you must ensure that it is always within your reach so you could safely stop it in case of an emergency.

General Instructions:

- In this document, the generic heading term "Activity" is used for actionable items as well as questions. These items must be specifically addressed in your written report.
- The symbol denotes an Activity that can be performed outside the laboratory at home prior to attending the lab, or after the lab if the activity requires answering questions based on your observations in the lab. You are strongly encouraged to complete these activities outside the laboratory hours to make efficient use of your lab time.
- The symbol denotes an Activity that must be carried out in the lab under TA supervision by powering the AEV from the LiPo battery.

1 Coordinate Frame Assignment in MacAEV

The coordinate frames in MacAEV are assigned according to Fig. 1. A schematic of the vehicle with its main components is given in Fig. 2.

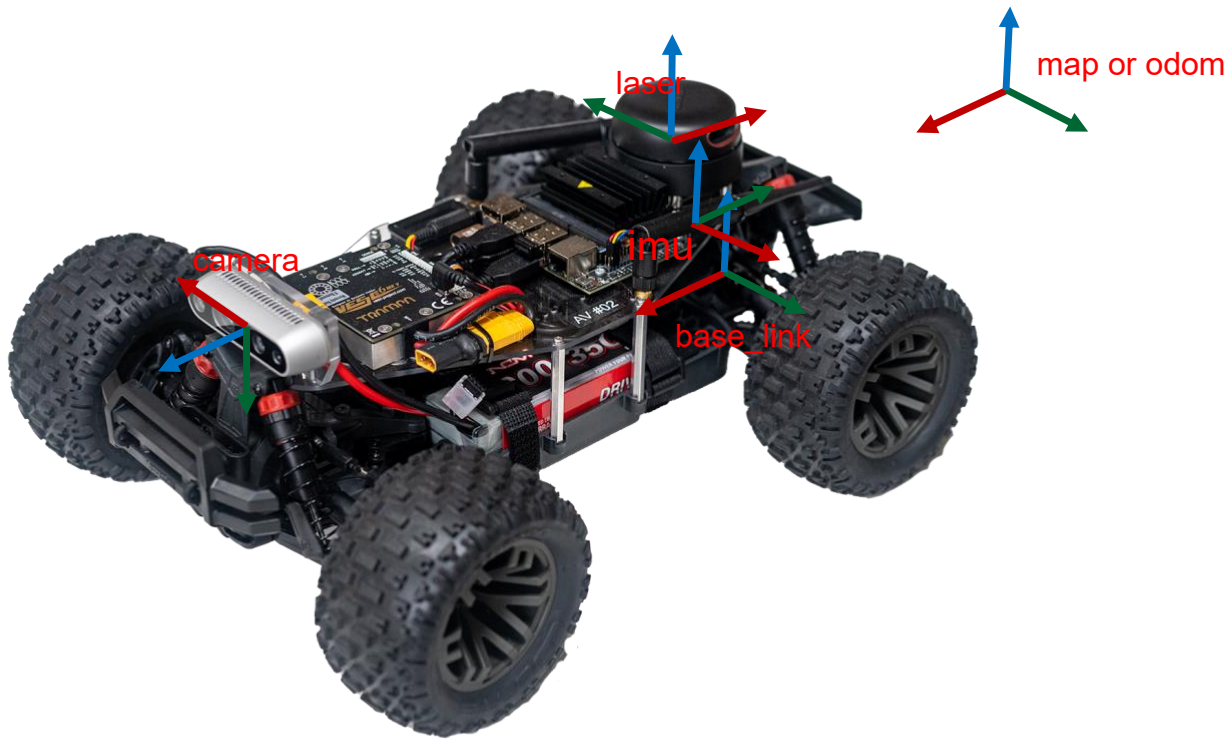

Figure 1: Coordinate Frame Assignment in MacAEV

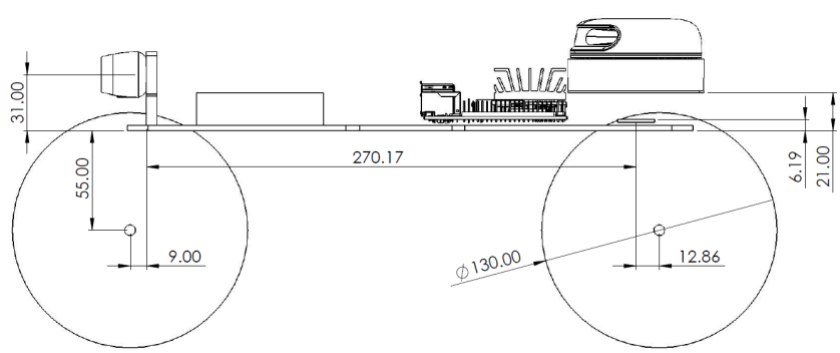

Figure 2: Schematic of MacAEV and its components placement

In Fig. 1, the frame “base_link” is the main reference frame on the vehicle and is attached to the middle of the rear axle. Its x axis points in the forward direction, and its z axis points in the upward direction. The frames of reference for the LiDAR, IMU, and Camera are denoted by “laser” (or “lidar”), “imu”, and “camera”, respectively.

Activity 1:

To accurately represent vehicle and environment data in ROS and rviz, you must define the relationships between different frames as shown in Fig. 1. Derive the fixed homogenous transformations that relate the frame "base_link" to the frames "laser", "imu", and "camera", i.e., $T_{laser}^{base_link}$, $T_{imu}^{base_link}$, $T_{camera}^{base_link}$. Using these results, complete the following ROS static transformation in the "experiment_incomplete.launch" file that is provided to you on Avenue to Learn by replacing unknown "???" symbols (see below) with appropriate values. Figs. 1&2 provide the required data for performing this task. Refer to the course lecture notes and ROS documentation for information on publishing static transformation in ROS. Replace the content of your existing "experiment.launch" file with the one you just completed. Include the updated file in your lab report submission.

```
<node pkg="tf2_ros" type="static_transform_publisher" name="base_link_to_imu"
      args="??? base_link imu" />

 <node pkg="tf2_ros" type="static_transform_publisher" name="base_link_to_laser"
      args="??? base_link laser" />

<node pkg="tf2_ros" type="static_transform_publisher" name="base_link_to_camera"
      args="??? base_link camera_link" />
```

2 BNO055 Inertial Measurement Unit

An Inertial Measurement Unit (IMU) measures motion variables used by many robotics navigation and localization algorithms. The MacAEV is equipped with the Bosch 9DOF[1] MEMS-based BNO055 IMU embedded in Adafruit interface board (See Fig. 3). The Bosch IMU fuses information from a gyroscope, an accelerometer, and a magnetometer to estimate the following motion variables:

- Absolute Orientation (4x1 Quaternion vector at 100 Hz)
- Angular Velocity (3x1 vector at 100 Hz)
- Linear Acceleration (3x1 vector at 100 Hz)

Activity 2:

In your report, explain how the orientation of a frame is expressed by the Quaternion vector.

[1] 9 DOF (degrees of freedom) refers to the fact that sensor fuses information from accelerometer (3 DOF), gyroscope for angular velocity (3 DOF) and magnetometer (3 DOF).

The BNO055 IMU provides a few other measurements which are not used in our project. You can find useful information about the Adafruit board here and the data sheet for BNO055 IMU chip here. The IMU device on the MacAEV is interfaced to the I2C Bus 1 on Jetson Nano J41 Header. You can find the pinout for J41 Header here. The Inter-Integrated Circuit (I^2C) is a serial protocol used for short-distance communication on a single device. It uses two lines, i.e., *data* and *clock* for communication.

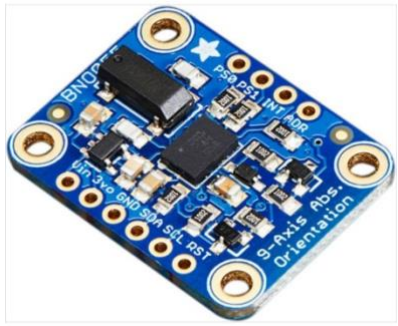

Figure 3: Adafruit IMU board utilizing Bosch BNO055 IMU chip

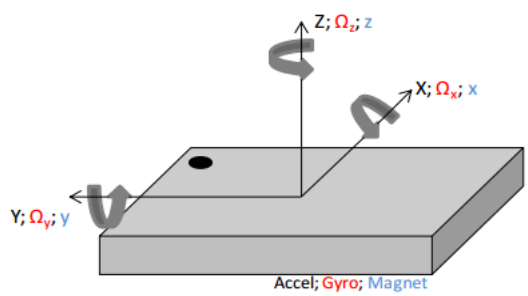

Figure 4: Default axis definition for BNO055 IMU

The default axes for the IMU measurement are defined in Fig. 4. The IMU frame assignment in Fig. 1 is based on this definition, the IMU chip position on the Adafruit board, and the board placement on MacAEV.

Activity 3:

We will use an open-source ROS driver to read the IMU data over the I2C bus. You can find and install this IMU driver on GitHub here.

- To install this driver, first go to the src folder in your catkin workspace and clone the GitHub repository using:

```
$ cd ~/catkin_ws/src
$ sudo git clone https://github.com/dheera/ros-imu-bno055
$ sudo git checkout 4c80f36
```

- Go back to the workspace root directory and build the new package located in the src folder by executing the following commands:

```
$ cd ~/catkin_ws
$ catkin_make
```

- After installing the driver, open and examine the content of the driver launch file.

```
$ roscd imu_bno055/launch/
$ sudo gedit imu.launch
```

The file content should look something like this:

```
<launch>
    <node ns="imu" name="imu_node" pkg="imu_bno055" type="bno055_i2c_node" respawn="true" respawn_delay="2">
        <param name="device" type="string" value="/dev/i2c-1"/>
        <param name="address" type="int" value="40"/> <!-- 0x28 == 40 is the default for BNO055 -->
        <param name="frame_id" type="string" value="imu"/>
    </node>
</launch>
```

Note that the device is mounted on /dev/i2c-1 with a default address of 0x28=40. The reference frame for the IMU data is the frame labeled as "imu" in Fig. 1 (see also Fig. 4).

The IMU driver publishes a "sensor_msgs/Imu" to the topic "/imu/data". This standard ROS message is defined as (see here for reference):

```
# This is a message to hold data from an IMU (Inertial Measurement Unit)
#
# Accelerations should be in m/s^2 (not in g's), and rotational velocity should be in rad/sec
#
# If the covariance of the measurement is known, it should be filled in (if all you know is the
# variance of each measurement, e.g. from the datasheet, just put those along the diagonal)
# A covariance matrix of all zeros will be interpreted as "covariance unknown", and to use the
# data a covariance will have to be assumed or gotten from some other source
#
# If you have no estimate for one of the data elements (e.g. your IMU doesn't produce an orientation
# estimate), please set element 0 of the associated covariance matrix to -1
# If you are interpreting this message, please check for a value of -1 in the first element of each
# covariance matrix, and disregard the associated estimate.

Header header

geometry_msgs/Quaternion orientation
float64[9] orientation_covariance # Row major about x, y, z axes

geometry_msgs/Vector3 angular_velocity
float64[9] angular_velocity_covariance # Row major about x, y, z axes

geometry_msgs/Vector3 linear_acceleration
float64[9] linear_acceleration_covariance # Row major x, y z
```

Activity 4:

The IMU driver reports *absolute* orientation of the IMU frame (see Figs. 1&4) with respect to a fixed earth-attached frame with axes, x (East), y (North) and z (Up). The orientation is reported in a *Quaternion* vector format. Run the IMU node, *imu_node*, by launching *imu.launch* from the *imu_bno055* package. Confirm that the IMU reported orientation is consistent with this convention, e.g., aligning the y-axis of the IMU frame with the direction of the earth north and

reading the value of the orientation. Recall that you can use the ROS command "rostopic" to echo the published content of a topic on the terminal screen. Use the ROS utility rosbag to record the IMU data as you manually move the vehicle. Upload this resulting .bag file onto the Avenue along with your report. Demonstrate to your TA in the lab that you have completed this task.

3 Localization with Wheel Odometry and IMU

Referring to Fig. 1, the goal is to compute the vehicle *configuration state* (position and orientation of the vehicle-attached frame "base_link"), also known as its *pose*, with respect to the fixed odometry frame "odom" (see Fig. 1). In Fig. 5, the vehicle pose is defined as a vector of three variables:

$$X = [x \quad y \quad \theta]^T \tag{1}$$

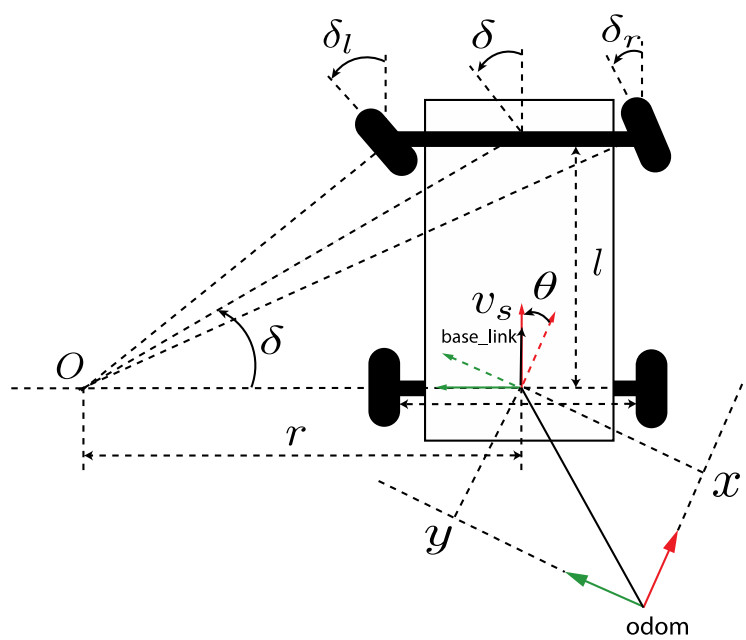

Figure 5: The vehicle schematics with "odom" (fixed to ground) and "base_link" (attached to vehicle) frames shown

Here, x and y are the coordinates of the origin of the frame "base_link" with respect to the Odometry frame "odom". The angular position of the vehicle, θ, is the rotation angle of the frame "base_link" around the z-axis of frame "odom". Recall from the lecture notes that the vehicle motion kinematics are given by the three first-order differential equations:

$$\dot{x} = v_s \cos\theta \tag{2}$$
$$\dot{y} = v_s \sin\theta \tag{3}$$
$$\dot{\theta} = \frac{v_s}{l} \tan\delta \tag{4}$$

The vehicle linear velocity v_s and steering angle δ are the control inputs. Assuming that at the starting time $t = 0$, the "base_link" and "odom" frames are aligned, i.e., $X(0) = [0 \quad 0 \quad 0]^T$, the vehicle pose at time t can be computed from the following integrals:

$$x(t) = \int_0^t v_s(\tau)\cos\big(\theta(\tau)\big)\,d\tau \tag{5}$$

$$y(t) = \int_0^t v_s(\tau)\sin\big(\theta(\tau)\big)d\tau \tag{6}$$

$$\theta(t) = \int_0^t \frac{v_s(\tau)}{l}\tan\big(\delta(\tau)\big)\,d\tau \tag{7}$$

These integrals are the basis for wheel odometry localization (dead reckoning), where the vehicle pose is computed from integrating of its kinematics equations, using the VESC-reported $v_s(t)$ and $\delta(t)$. In practice, the above integrals are computed at discrete sample times. Using the *Euler's* integration method, these integrals can be approximated by following pose update equations in discrete time:

$$x_{k+1} = x_k + \delta t_k v_{s_k} cos\,(\theta_k) \tag{8}$$

$$y_{k+1} = y_k + \delta t_k v_{s_k}\, sin(\theta_k) \tag{9}$$

$$\theta_{k+1} = \theta_k + \delta t_k \frac{v_{s_k}}{l} tan\,(\delta_k) \tag{10}$$

$$x_0 = 0, \qquad y_0 = 0, \qquad \theta_0 = 0 \tag{11}$$

where the subscript k refers the value of the variable at the k'th sample time, and δt_k is the time step.

MacAEV is equipped with an on-board IMU that measures the vehicle heading (orientation) angle with respect to an earth-attached frame with axes x (east) y (north) and z (up). Therefore, we can simply skip the angular integration in (10) and directly obtain the vehicle orientation angle from the IMU yaw measurement.

Remark: Recall the "zero" reference for IMU angle measurement is different from that of the vehicle orientation angle, θ. You should record the IMU yaw angle measurement at time $t = 0$ and subtract this offset value from all its future measurements to correctly account for this change of reference for angle measurement.

Activity 5:

In this activity, you modify your existing "experiment.cpp" file from Lab 5 to implement the wheel odometry algorithm described above, using the IMU measurements for the vehicle orientation angle. Copy the contents of the file "experiment_incomplete.cpp" to "experiment.cpp". In your implementation:

- The fixed frame "odom" should be set to the vehicle starting pose ("base_link") at time $t = 0$. In practice, this simply means that the vehicle starting pose is given by $X(0) = [0 \quad 0 \quad 0]^T$.

- Record the IMU yaw angle reading at $t = 0$; subtract this offset value from future IMU yaw angle measurements to obtain the vehicle rotation angle directly from the IMU.
- Compute the vehicle x, y position at each time step using the discrete-time integration equations in (8)-(9).
- Publish the wheel odometry results as a ROS *topic* as well as a ROS *transformation*. It is good practice to pass the names of the two frames "odom" and "base_link" to the experiment.cpp node via two ROS parameters in the file "params.yaml", as opposed to hardcoding them in the file directly. ROS uses a standard Odometry Message type for communicating odometry information. See the link here for the definition of this message and the link here for an example of how to publish an Odometry Message and a ROS Transformation. You should be already familiar with the ROS *tf* system intended for defining relationships among coordinate frames in the ROS system.
- You are provided a "params.yaml" file on Avenue to learn. Use this file as a reference to make sure you add the necessary parameters for Lab 6 to your package's "params.yaml". **Do not overwrite your calibration parameters from Lab 5**.
- After completing the file "experiment.cpp", go to the root of the catkin workspace and re-build the package.

Include a copy of the modified "experiment.cpp" with your report.

Activity 6:

Start control code by launching "experiment.launch". Run the ROS visualization tool "rviz", set the fixed frame to "odom" (or the name you chose for the odometry frame) and add a TF display. Select the two frames "odom" (fixed to the ground) and "base_link" (attached to the vehicle) for display. The two frames should be initially aligned before you start driving the vehicle (why?). Mark the vehicle initial pose on the floor. Drive the vehicle manually using the joystick; the frame "base_link" should start moving, showing the vehicle current pose. After driving the vehicle around for some time, re-position it at its original pose on the floor. Ideally, the two frames "odom" and "base_link" should be re-aligned at this point. However, this is unlikely to be the case in practice, particularly as the vehicle overall traveling distance increases. Discuss potential sources of errors in wheel-based odometry computations that may cause a *drift* in the computed pose of the vehicle. What do you expect the impact of using IMU measurement for yaw angle to be on the localization errors? Demonstrate to your TA in the lab that you have completed this activity. Use the rosbag utility to record the vehicle pose during the experiment and include this file as part of your lab report submission.

4 Mapping using LiDAR and Wheel Odometry Data

So far, we have localized the vehicle by finding its pose with respect to the fixed "odom" frame using VESC and IMU data. In many mobile robotics applications, a map of the environment is required for tasks such as path planning and navigation. The environment map provides useful information that the robot can utilize to plan its path and achieve its objectives more quickly and

efficiently. Search and rescue robots, for example, can benefit significantly from having a prior map of their task environment to use in mission planning. Autonomous cars use available maps of public roads for navigation, and update these maps in real-time to account for other vehicles and road users in their path planning algorithms. While in some applications the map of the environment may be (partially) known a priori (e.g., driving on public roads), there are many others in which the environment map is unknown and must be constructed from data collected by sensors in real time.

4.1 Occupancy Grid Map

Occupancy Grid Maps are popular in mobile robotics applications due to their simplicity and ease of use. An Occupancy Grid Map represents the environment in discretized cells the values of which are the probabilities of the cells being occupied by obstacles. Fig. 6 shows the vehicle moving over a two-dimensional (2D) discretized grid representation of the environment. The fixed frame "odom" is the reference frame for the map. The "base" and "lidar" frames are attached to the centers of vehicle rear axle and LiDAR, respectively. As the vehicle scans the environment, it detects obstacle points (A, B, C, …) with respect to the "lidar" frame. The LiDAR returns obstacle ranges at discretized angles. If the vehicle pose with respect to the "odom" frame is known (e.g., based on wheel odometry), then the detected obstacle points can be projected back onto the "odom" frame and grid cells. A map of the environment can be gradually constructed over time as more LiDAR scan data becomes available.

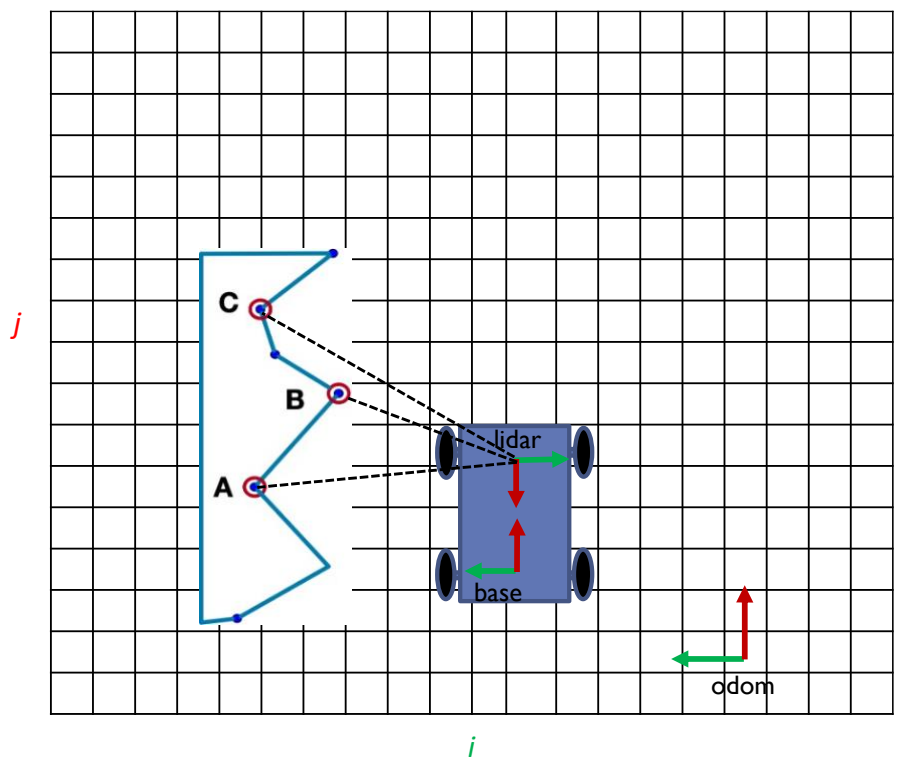

Figure 6: A vehicle equipped with LiDAR moves in 2D environment represented by an Occupancy Grid. LiDAR detects obstacle points (A, B, C, …) with respect to its frame of reference "lidar". Occupancy Grid Map is constructed with respect to fixed frame "odom" using LiDAR scan and wheel odometry data.

Let $p(m_{ij})$ be the probability of the map cell in position (i,j) being occupied by an object. In general,

$$\begin{cases} \text{if } p(m_{ij}) \geq p_{oc} > 0.5 & \text{cell is considered occupied} \\ \text{if } p_{fr} < p(m_{ij}) < p_{oc} & \text{cell status is considered uknown} \\ \text{if } p(m_{ij}) \leq p_{fr} < 0.5 & \text{cell is considered free} \end{cases}$$

The Occupancy Grid Mapping algorithm finds the cell occupancy probabilities based on all the LiDAR scan and wheel odometry data from the initial time t_0 up to the current time t_k, denoted as

$$p(m_{ij}|z_{0:k}, x_{0:k}) \tag{13}$$

Here,

$$z_{0:k} = \{z_0, z_1, \cdots, z_k\} \tag{14}$$

is the set of all LiDAR scan data from the initial time t_0 to the current time t_k , and $z_j = \left\{q_n^{l_j}\right\}$ is the set of LiDAR scan data points $q_n^{l_j}$ at time t_j. Furthermore,

$$x_{0:k} = \{x_0, x_1, \cdots, x_k\} \tag{15}$$

is the set of *known* vehicle poses from the initial time t_0 to the current time t_k, where

$$x_k = (\theta_k,\ d_{b_k}^o) \tag{16}$$

and θ_k and $d_{b_k}^o$ are the vehicle yaw rotation angle and position vector with respect to the fixed map frame "odom" at time step t_k.

4.2 Inverse Sensor Model

An *Inverse Sensor* model is needed for the development of the mapping algorithm. The inverse sensor model produces the cell occupancy probabilities given the *current* vehicle pose x_k and the LiDAR scan data z_k, i.e., $p(m_{ij}|z_k, x_k)$. Referring to Fig. 7, for each cell m_{ij} in the map, we find the *closest* LiDAR ray to the line connecting the center of the cell to origin of the LiDAR frame. Conditioned on the current LiDAR scan data and vehicle pose, the cell is considered occupied (red) if the LiDAR return from the closest ray falls inside the cell; it is considered free (green) if the LiDAR return is at a further distance from the origin of LiDAR frame than that of the center of the cell; otherwise, the status of cell is unknown (blue). Note that Fig. 7 shows only those cells for which the given LiDAR ray happens to be the closest ray. The cell conditional occupancy probabilities are then given by:

$$p(m_{ij}|z_k, x_k) = \begin{cases} p_{oc} > 0.5 & \text{occupied} \\ p_{fr} < 0.5 & \text{free} \\ p_{un} = 0.5 & \text{unknown} \end{cases} \tag{17}$$

This Inverse Sensor model assumes that the grid cell occupancies are *independent* of each other in a probabilistic sense.

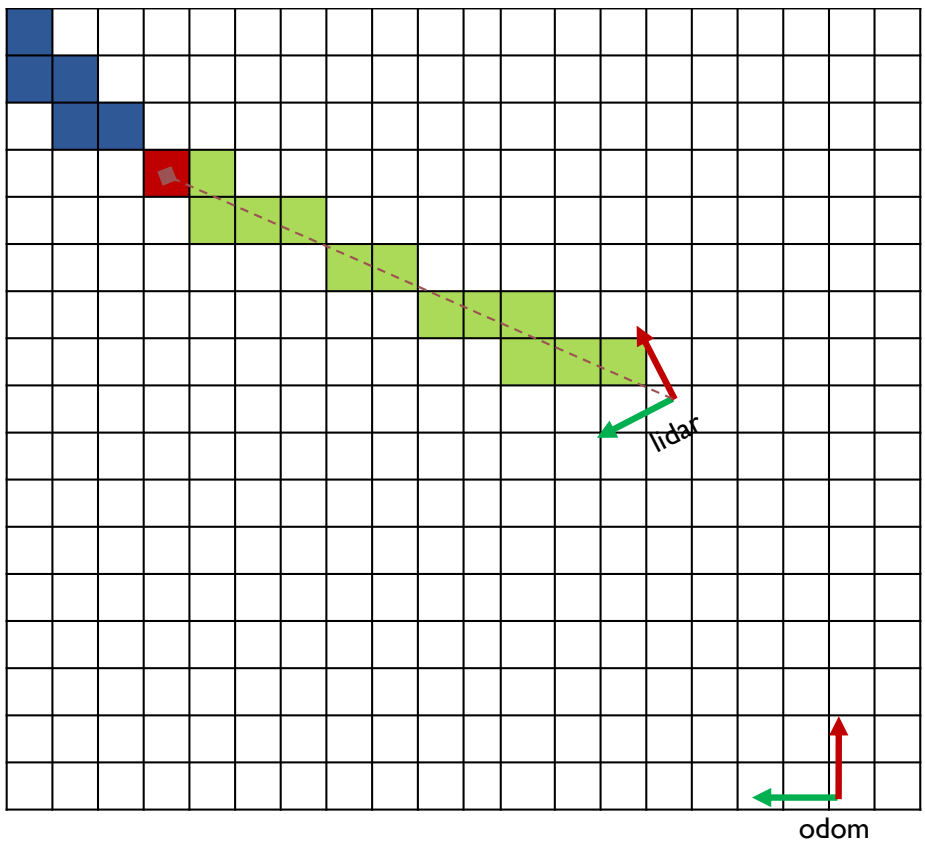

Figure 7: Inverse Sensor Model: Free (green), Occupied (red) and Unknown (blue) cells based on one LiDAR return

4.3 Recursive Update of Cell Occupancy Probabilities

Further assuming that the obstacles in the environment are *static*, it can be shown that (see course lecture slides for details),

$$\frac{p(m_{ij}|z_{0:k},\, x_{0:k})}{1 - p(m_{ij}|z_{0:k},\, x_{0:k})} = \frac{p(m_{ij}|z_k, x_k)}{1 - p(m_{ij}|z_k, x_k)} \cdot \frac{p(m_{ij}|z_{0:k-1},\, x_{0:k-1})}{1 - p(m_{ij}|z_{0:k-1},\, x_{0:k-1})} \cdot \frac{1 - p(m_{ij})}{p(m_{ij})} \tag{18}$$

Here $p(m_{ij})$ represents any available *prior knowledge* about cell occupancies.

Now, let

$$l(y) = \log \frac{p(y)}{1 - p(y)} \tag{19}$$

$$p(y) = 1 - \frac{1}{1 + e^{l(y)}} \tag{20}$$

By taking the log of the expression in (18), one can write

$$l(m_{ij}|z_{0:k}, x_{0:k}) = \underbrace{l(m_{ij}|z_k, x_k)}_{inverse\ sensor\ model} + \underbrace{l(m_{ij}|z_{0:k-1}, x_{0:k-1})}_{recursive\ term} - \underbrace{l(m_{ij})}_{prior} \tag{21}$$

Note that the first term on the right side is computed from the Inverse Sensor model in (17) and the second term is the cell occupancy probability from the previous time step t_{k-1}. In most cases, there is no prior information about the cell occupancies, i.e.,

$$l(m_{ij}) = \log \frac{p_{un}}{1 - p_{un}} = \log \frac{0.5}{1 - 0.5} = 0 \tag{22}$$

The Occupancy Grid Mapping algorithm is summarized in Table 1 below.

Table 1: 2D Occupancy Grid Mapping Algorithm with LiDAR Scan Data and Known Vehicle Pose	
	Inputs: LiDAR scan data z_k and vehicle known pose x_k at time t_k **Outputs**: cell occupancy probabilities $p(m_{ij}\|z_{0:k}, x_{0:k})$ At time t_0, initialize the cell occupancy probabilities to $p(m_{ij}) = 0.5$ *for each grid cell* m_{ij}*:* • Find the closest LiDAR ray to the line connecting the center of the cell m_{ij} to the center of the LiDAR frame on the vehicle. • Inverse Sensor Model: Determine the probability of cell occupancy based ONLY on the current LiDAR return from the closest ray and the current vehicle pose, i.e., $p(m_{ij}\|z_k, x_k)$ *in* (17) *and* $l(m_{ij}\|z_k, x_k)$: **Occupied**: LiDAR return falls inside the cell; $p(m_{ij}\|z_k, x_k) = p_{oc}$ **Free**: the LiDAR return range is greater than the cell range; $p(m_{ij}\|z_k, x_k) = p_{fr}$ **Unknown**: the LiDAR return range is less than the cell range; $p(m_{ij}\|z_k, x_k) = p_{un}$ • Update the cell occupancy $l(m_{ij}\|z_{0:k}, x_{0:k})$ using the recursive equation in (21) for its log odds

Remark: The LiDAR returns are reported with respect to the "lidar" (or "laser") frame. The vehicle pose from wheel odometry describes vehicle attached frame "base" (or "base_link") with respect to the world frame "odom". The relationship between "lidar" and "base" frames is fixed and known. This information may be helpful in the calculations of the Occupancy Grid Mapping algorithm in Table 1.

4.4 Occupancy Grid Map Representation in ROS

ROS uses the data type "nav_msgs/OccupancyGrid" to represent Occupancy Grid Map data. This message type is defined as:

```
Header header
uint32 seq
time stamp
string frame_id

MapMetaData info
time map_load_time
float32 resolution     // width of each grid cell in meters
uint32 width           // number of columns in occupancy grid
uint32 height.         // number of rows in occupancy grid

geometry_msgs/Pose origin

geometry_msgs/Point position        // x & y positions of cell (0,0)
float64 x
float64 y
float64 z                           // not used in 2D maps
geometry_msgs/Quaternion orientation
float64 x
float64 y
float64 z
float64 w
int8[] data                         // occupancy probabilities
```

When creating an instance of the Occupancy Grid Map, you should:

1. Assign the name of the fixed map frame (e.g., "odom" or "map") to the variable: `header.frame_id`.
2. Assign the current time, `rospy.Time.now()` in Python or `ros::Time::now()` in C++, to the variable: `header.stamp`.
3. Assign the map cell size/resolution (in meters) to the variable `info.resolution`.
4. Assign the number of columns and rows in the Occupancy Grid to the variables `info.width` and `info.height`, respectively.
5. Assign the x and y coordinates of the cell (0,0) (lower left cell) in the Occupancy Grid with respect to the fixed map frame "odom" to the variables `info.origin.position.x` and `info.origin.position.y`, respectively.

6. Assign the orientation of the Occupancy Grid with respect to the fixed map frame using a Quaternion representation in variables `info.origin.orientation.x`, `info.origin.orientation.y`, `info.origin.orientation.z`, `info.origin.orientation.w`. Orientation angle is usually set to zero, i.e., it has a $(x = 0, y = 0, z = 0, w = 1)$ Quaternion representation.

The map data is stored in a row-major order in "data". The occupancy probabilities are expressed in the range [0-100] with a value of -1 denoting an unknown status. In your implementation, you should assign the cell value to 100 if $p(m_{ij}|z_{0:k}, x_{0:k}) > p_{oc}$, to 0 if $p(m_{ij}|z_{0:k}, x_{0:k}) < p_{fr}$, and to -1, otherwise. This assignment divides the map cells into three distinct categories of *occupied*, *free*, or *unknown* cells. Note that the x and y values increment by the cell resolution with increasing column and row indices, respectively.

Activity 7:

Develop a new ROS node under the f1tenth_simulator package to implement an Occupancy Grid Map using the Algorithm in Table 1. You have the option of implementing the Occupancy Grid Map node in Python or C++. You are provided with a template Python file "occupanctygridmap_incomplete.py" on Avenue as a starting point for a Python-based implementation. Refer to the ROS online documentation for information on how to create ROS nodes in Python or C++.

Your Occupancy Grid Map node must accomplish the following tasks:

1- Read the map parameters (e.g., width, height, resolution, etc.) from the parameters file "params.yaml" and instantiate a Grid Occupancy message using these parameters. Ensure you add the map parameters to your "params.yaml" file.
2- Subscribe to the existing wheel odometry and laser scan topics to receive odometry (vehicle pose) and LiDAR scan data as they become available on these topics.
3- Initialize the Occupancy Grid Map cell occupancy data to "unknown" status (i.e., $p(m_{ij}) = 0.5$).
4- Define two callback functions `odom_callback()` and `lidar_callback()` triggered by the messages published on the Odometry and LiDAR Scan topics.
5- In `odom_callback()`, read the vehicle's current pose with respect to the fixed "odom" frame. The fixed "odom" frame is also the reference frame for the Occupancy Grid Map.
6- In `lidar_callback()`, using the most up-to-date vehicle pose from `odom_callback()`, update the cell occupancy probabilities $p(m_{ij}|z_{0:k}, x_{0:k})$ with the received LiDAR scan data according to the Algorithm in Table 1.
7- In `lidar_callback()`, use $p(m_{ij}|z_{0:k}, x_{0:k})$ to publish an updated Occupancy Grid Map to the map topic.

Include the code of your completed Occupancy Grid Mapping algorithm with your submission.

Activity 8:

The rosbag file named "mapping_experiment.bag" on Avenue contains a recording of the topics /scan and /odom during the experiment depicted below. The experiment involves driving the MacAEV down the ITB hallway from the starting position illustrated in Figure 8, past the initial set of boxes, and returning to the final position shown in Figure 9. The entire experiment was conducted over approximately 46 seconds.

You should test your occupancy grid mapping node without driving the vehicle using this rosbag file. The rosbag file will publish to the /scan and /odom topics as if you were driving the vehicle. You will need a modified launch file that publishes the static coordinate transformations and only launches the occupancy grid mapping node. Other nodes do not need to be launched. Use the rosbag utility to record the map data as being constructed. Include this data with your submission.

Figure 8: Vehicle initial pose

Figure 9: Vehicle final pose

Activity 9:

Add the new Occupancy Grid Map node to the launch file "experiment.launch". You are now ready to test your Occupancy Grid Mapping algorithm with the vehicle driving in the hallways. Place a few cardboard boxes as obstacles in the hallway along the vehicle path. Launch "experiment.launch" to start the mapping algorithm. Also launch the ROS visualization tool rviz to display the map as being constructed. Set the rviz fixed frame to the fixed frame for the Occupancy Grid Map (i.e., "odom") and add displays of TF and Map types from the left panel. Deselect all frames except for **odom** (fixed frame) and **base_link** (body-attached frame) under TF. Mark the initial vehicle pose on the floor. Manually drive the vehicle around using the Joystick and observe the map of environment as well as the vehicle pose. The vehicle should return to its initial pose at the end of the experiment. If needed adjust the map parameters (e.g., width, height, resolution, p_{oc}, p_{fr}, etc.) and repeat the experiment until you are satisfied with the resulting map. Briefly discuss the impact of these parameters on the mapping results in your report.

You must use the rosbag utility to record the data published on select topics related to Odometry, ROS coordinate transformation system (/tf and /tf_static) and the Occupancy Grid Map. Upload the ROS bag file onto the Avenue along with your report. Comment on your observations and particularly on the sources of mapping errors. What happens if you have moving obstacles in the environment? Note that this technically violates the assumption of a static environment under which the Occupancy Grid Mapping algorithm was developed.

Activity 10:

Write a closing statement summarizing what you have accomplished in this lab. Comment on the limitations of the localization and mapping algorithms you have developed. Suggest ways in which you can potentially address some of these limitations.